

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №3

По дисциплине: «Обработка изображений в интеллектуальных системах»
Тема: «Обучение детекторов объектов»

Выполнил:
Студент 4 курса
Группы ИИ-25
Заседатель Н.С.
Проверила:
Андренко К.В.

Брест 2026

Цель работы: осуществлять обучение нейросетевого детектора для решения задачи обнаружения заданных объектов.

Задание:

1. Базируясь на своем варианте, ознакомится с выборкой для обучения детектора, выполнить необходимые преобразования данных для организации процесса обучения (если это нужно!);
2. Для заданной архитектуры нейросетевого детектора организовать процесс обучения для своей выборки. Оценить эффективность обучения на тестовой выборке (mAP);
3. Реализовать визуализацию работы детектора из пункта 1 (обнаружение знаков на отдельных фотографиях из сети Интернет);
4. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

В-т	Детектор	Датасет
6	YOLO11m	Камень-ножницы-бумага: https://universe.roboflow.com/roboflow-58fyf/rock-paper-scissors-scissors-dataset/14

Ход работы:

1. Для реализации проекта был выбран датасет для распознавания жестов «Камень, ножницы, бумага». Выборка состоит из изображений трех классов объектов:

- **Rock** (Камень) – ладонь, сжатая в кулак;
- **Scissors** (Ножницы) – выставленные указательный и средний пальцы;
- **Paper** (Бумага) – полностью раскрытая ладонь.

Все изображения в выборке имеют соответствующую разметку в формате YOLO, где для каждого объекта заданы координаты ограничивающей рамки (bounding box) и индекс класса.

Структура датасета разделена на две основные подвыборки:

- **train** (обучающая выборка) – для подстройки весов нейросети;
- **val** (валидационная/тестовая выборка) – для контроля переобучения и оценки точности.

В корне датасета находится конфигурационный файл data.yaml, который связывает пути к изображениям и текстовым файлам разметки, а также хранит имена классов: names: ['Paper', 'Rock', 'Scissors']. Для организации стабильного и быстрого процесса обучения в облачной среде Google Colab были проведены следующие технические преобразования:

- **Архивация и пакетная загрузка:** Исходный датасет, состоящий из тысяч отдельных файлов изображений и текстовых аннотаций, был предварительно упакован в единый архив dataset.zip. Это позволило обойти ограничение облачного сервера на скорость загрузки множества мелких файлов и сократило время копирования данных с локального компьютера в десятки раз.
- **Программная подготовка структуры:** На стороне сервера Google Colab была выполнена автоматическая распаковка архива с помощью терминальной команды: `!unzip dataset.zip -d dataset`, параметр `-d dataset` гарантировал строгое соблюдение иерархии папок, требуемой для архитектуры YOLO, изолировав данные в рабочей директории без нарушения относительных путей.
- **Адаптация путей разметки:** Проверено соответствие путей внутри конфигурационного

файла data.yaml текущей файловой системе виртуальной машины (/content/dataset/...), что исключило возникновение ошибок отсутствия файлов (FileNotFoundError) при инициализации загрузчика данных (dataloader).

2. В качестве архитектуры нейросетевого детектора была выбрана современная модель YOLO11m (средний размер – medium), обеспечивающая оптимальный баланс между скоростью работы и точностью распознавания объектов. Процесс обучения был организован в облачной среде Google Colab с аппаратным ускорением на видеокарте Tesla T4. В ходе настройки гиперпараметров были установлены следующие ключевые конфигурации:

- **epochs** = 50 – общее количество циклов прогона всей обучающей выборки через нейросеть;
- **imgsz** = 640 – размер входных изображений (масштабирование до 640x640 пикселей);
- **batch** = 16 – размер пакета данных, одновременно загружаемого в видеопамять для одной итерации обновления весов.

После завершения всех 50 эпох обучения была проведена финальная валидация на отложенной тестовой (валидационной) выборке для расчёта метрик средней точности (mAP – Mean Average Precision). Модель показала крайне высокую эффективность детектирования жестов. Итоговые метрики качества составили:

- **mAP50** = 0.9378 (93.78%) – метрика точности при пороге пересечения (IoU) равном 0.5. Показывает, что в ~94% случаев нейросеть безошибочно находит жест на изображении и правильно определяет его класс.
- **mAP50-95** = 0.7499 (74.99%) – интегральная метрика точности, рассчитываемая при строгих порогах IoU от 0.5 до 0.95. Высокое значение доказывает, что модель очень точно строит ограничивающие рамки (bounding boxes) вокруг рук, минимизируя отклонения.

Поклассовая эффективность распознавания жестов распределилась следующим образом:

- **Paper** (Бумага): mAP50 = 0.938
- **Rock** (Камень): mAP50 = 0.929
- **Scissors** (Ножницы): mAP50 = 0.946

Динамика изменения функций потерь (включая Distribution Focal Loss и Box Loss) показала стабильное падение на протяжении всех эпох, а метрики точности вышли на плато к концу обучения, что свидетельствует об успешной сходимости алгоритма и отсутствии явного переобучения (overfitting). Обученная модель полностью пригодна для практического использования.

3. Для выполнения этого пункта задания обученные в облаке веса нейросети (best.pt) были экспортированы и развернуты локально на персональном компьютере. В среде разработки VS Code был реализован автономный скрипт на языке Python, использующий библиотеки ultralytics (для работы с моделью) и opencv-python (для сохранения графических файлов). Основная задача скрипта – автоматизировать пакетную обработку: брать все изображения из заданной папки по очереди, прогонять их через нейросеть, отрисовывать bounding box'ы (рамки объектов) с названиями классов и сохранять результат.

В ходе первоначального запуска скрипта со стандартным порогом уверенности (conf=0.4) модель пропустила часть тестовых домашних фотографий (вывод: no detections). Это обусловлено тем, что реальные фотографии содержали уникальные внешние шумы, которых

не было в идеальных студийных изображениях обучающего датасета. Для решения этой проблемы порог отсечения был скорректирован и зафиксирован на уровне conf=0.15 (15% уверенности). Это позволило детектору успешно идентифицировать жесты в условиях сложного фона.

Результаты работы алгоритма:

- Модель успешно справилась с поиском как одиночных жестов, так и групповых кадров, содержащих несколько рук одновременно (например, сцена дуэли «Камень против Бумаги»).
- На обработанных изображениях в папке results нейросеть безошибочно выстроила ограничивающие рамки вокруг ладоней и наложила текстовые метки классов (например, Paper 0.71, что говорит о 71% уверенности в правильности жеста «Бумага» на данном кадре).
- Процесс обработки одного кадра на центральном процессоре (CPU) ноутбука занимает в среднем 200-240 мс, что подтверждает высокую скорость работы архитектуры YOLO даже без использования дискретной видеокарты на этапе предсказания.

Результат:

`!nvidia-smi`

Tue Sep 22 16:42:29 2026

+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										
NVIDIA-SMI 580.82.07			Driver Version: 580.82.07				CUDA Version: 13.0			
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										
GPU Name		Persistence-M		Bus-Id		Disp.A		Volatile Uncorr. ECC		
Fan Temp Perf		Pwr:Usage/Cap				Memory-Usage		GPU-Util Compute M.		
								MIG M.		
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										
0 Tesla T4		Off		00000000:00:04.0 Off				0		
N/A 60C P8		10W / 70W		0MiB / 15360MiB				0% Default		
								N/A		
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+										

Processes:

GPU		GI		CI		PID		Type		Process name		GPU Memory	
		ID		ID								Usage	
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													
No running processes found													
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+													

`!pip install ultralytics`

... Collecting ultralytics

Downloading ultralytics-8.4.159-py3-none-any.whl.metadata (46 kB)

46.7/46.7 kB 2.5 MB/s eta 0:00:00

Requirement already satisfied: cloudpickle>=3.1.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.1.2)

Requirement already satisfied: filelock>=3.16.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.32.5)

Requirement already satisfied: numpy>=1.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.1.3)

Requirement already satisfied: matplotlib>=3.3.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (3.10.0)

Requirement already satisfied: opencv-python!>=4.13.0.90,>=4.7.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.0.0.93)

Requirement already satisfied: pillow>=7.1.2 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (11.3.0)

Requirement already satisfied: pyyaml>=5.3.1 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (6.0.3)

Requirement already satisfied: requests>=2.23.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.32.4)

Requirement already satisfied: torch>=1.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (2.11.0+cu128)

Requirement already satisfied: torchvision>=0.9.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (0.26.0+cu128)

Requirement already satisfied: psutil>=5.8.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (5.9.5)

Requirement already satisfied: polars>=0.20.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (1.35.2)

Requirement already satisfied: nvidia-ml-py>=12.0.0 in /usr/local/lib/python3.13/dist-packages (from ultralytics) (13.610.43)

Collecting ultralytics-thop>=2.1.6 (from ultralytics)

Downloading ultralytics_thop-2.1.6-py3-none-any.whl.metadata (13 kB)

Collecting ultralytics-platform>=0.1.45 (from ultralytics)

```
from ultralytics import YOLO
```

```
model = YOLO("yolo11m.pt")
```

```
model.train(
    data="dataset/data.yaml",
    epochs=50,
    imgsz=640,
    batch=16
)
```

Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo11m.pt> to 'yolo11m.pt': 100% 38.8MB 171.9MB/s 0.2s
ultralytics 8.4.159 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
engine/trainer: agnostic_nms=False, amp=True, angle=1.0, augment=False, auto_augment=randaugment, batch=16, bgr=0.0, box=7.5, cache=False, cfg=None, channels_last=None, cla
Downloading <https://ultralytics.com/assets/Arial.ttf> to '/root/.config/Ultralytics/Arial.ttf': 100% 755.1KB 30.5MB/s 0.0s
Overriding model.yaml nc=80 with nc=3

	from	n	params	module	arguments
0	-1	1	1856	ultralytics.nn.modules.conv.Conv	[3, 64, 3, 2]
1	-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
2	-1	1	111872	ultralytics.nn.modules.block.C3k2	[128, 256, 1, True, 0.25]
3	-1	1	590336	ultralytics.nn.modules.conv.Conv	[256, 256, 3, 2]
4	-1	1	444928	ultralytics.nn.modules.block.C3k2	[256, 512, 1, True, 0.25]
5	-1	1	2360320	ultralytics.nn.modules.conv.Conv	[512, 512, 3, 2]
6	-1	1	1380352	ultralytics.nn.modules.block.C3k2	[512, 512, 1, True]
7	-1	1	2360320	ultralytics.nn.modules.conv.Conv	[512, 512, 3, 2]
8	-1	1	1380352	ultralytics.nn.modules.block.C3k2	[512, 512, 1, True]
9	-1	1	656896	ultralytics.nn.modules.block.SPPF	[512, 512, 5]
10	-1	1	990976	ultralytics.nn.modules.block.C2PSA	[512, 512, 1]
11	-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']

Remapped 1/3 cls head rows from pretrained weights by class name
Transferred 649/649 items from pretrained weights
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks...
AMP: downloading yolo26n.pt for AMP checks (one-time, not used for training)...
Downloading <https://github.com/ultralytics/assets/releases/download/v8.4.0/yolo26n.pt> to 'weights/yolo26n.pt': 100% 5.3MB 94.7MB/s 0.1s
AMP: checks passed
train: Fast image access (ping: 0.0±0.0 ms, read: 1231.3±714.3 MB/s, size: 41.2 KB)
train: Scanning /content/dataset/train/labels... 6455 images, 2516 backgrounds, 0 corrupt: 100% 6455/6455 2.1Kit/s 3.1s
train: New cache created: /content/dataset/train/labels.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01, method='weighted_average', num_output_channels
val: Fast image access (ping: 0.0±0.0 ms, read: 1120.4±547.5 MB/s, size: 32.0 KB)
val: Scanning /content/dataset/valid/labels... 576 images, 238 backgrounds, 0 corrupt: 100% 576/576 2.2Kit/s 0.3s
val: New cache created: /content/dataset/valid/labels.cache
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'optimizer', 'lr' and 'momentum' automatically...
optimizer: AdamW(lr=0.001429, momentum=0.9) with parameter groups 106 weight(decay=0.0), 113 weight(decay=0.0005), 112 bias(decay=0.0)
Plotting labels to /content/runs/detect/train-3/labels.jpg...
Using 6455 train, 576 val images for fraction=1.0 at imgsz=640
Using 2 dataloader workers
Logging results to /content/runs/detect/train-3
Starting training for 50 epochs...

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/50	7.87G	1.539	2.485	1.787	14	640: 100% 404/404 2.1it/s 3:09
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 1.5it/s 12.3s
	all	576	400	0.114	0.323	0.102 0.0409
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/50	7.98G	1.698	2.587	1.904	9	640: 100% 404/404 2.3it/s 2:54
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 2.4it/s 7.4s
	all	576	400	0.329	0.396	0.278 0.145
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/50	8.03G	1.649	2.512	1.86	9	640: 100% 404/404 2.3it/s 2:53
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 2.3it/s 7.8s
	all	576	400	0.612	0.272	0.358 0.181
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
4/50	7.98G	1.554	2.261	1.767	11	640: 100% 404/404 2.3it/s 2:54
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 2.4it/s 7.7s
	all	576	400	0.514	0.416	0.452 0.262
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
5/50	7.97G	1.45	2.024	1.684	10	640: 100% 404/404 2.3it/s 2:57
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 2.5it/s 7.2s
	all	576	400	0.523	0.481	0.464 0.264
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
6/50	7.97G	1.388	1.851	1.633	8	640: 100% 404/404 2.4it/s 2:52
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 18/18 2.4it/s 7.4s
	all	576	400	0.573	0.663	0.667 0.414

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
45/50	7.97G	0.5793	0.3691	1.041	5	640: 100%  404/404 2.4it/s 2:50	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.4it/s 7.5s
	all	576	400	0.926	0.945	0.942	0.749
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
46/50	7.97G	0.5668	0.3514	1.028	3	640: 100%  404/404 2.4it/s 2:52	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.4it/s 7.4s
	all	576	400	0.939	0.936	0.939	0.748
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
47/50	7.98G	0.5519	0.3446	1.02	4	640: 100%  404/404 2.4it/s 2:51	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.5it/s 7.1s
	all	576	400	0.943	0.934	0.943	0.75
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
48/50	7.97G	0.5374	0.3372	1.009	5	640: 100%  404/404 2.4it/s 2:49	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.5it/s 7.2s
	all	576	400	0.94	0.928	0.938	0.75
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
49/50	7.97G	0.5239	0.3312	0.9989	6	640: 100%  404/404 2.4it/s 2:49	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.4it/s 7.5s
	all	576	400	0.955	0.91	0.943	0.748
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size	
50/50	7.98G	0.515	0.3216	0.9902	10	640: 100%  404/404 2.4it/s 2:51	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%  18/18 2.4it/s 7.6s
	all	576	400	0.942	0.919	0.94	0.749

50 epochs completed in 2.541 hours.

Optimizer stripped from /content/runs/detect/train-3/weights/last.pt, 40.5MB

Optimizer stripped from /content/runs/detect/train-3/weights/best.pt, 40.5MB

Validating /content/runs/detect/train-3/weights/best.pt...

Ultralytics 8.4.159 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

YOLO11m summary (fused): 125 layers, 20,032,345 parameters, 0 gradients, 67.8 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95)	100%	
all	576	400	0.941	0.927	0.938	0.749		18/18 1.5it/s 12.1s
Paper	132	139	0.949	0.892	0.938	0.754		
Rock	121	141	0.933	0.957	0.929	0.724		
Scissors	116	120	0.941	0.932	0.946	0.767		

Speed: 1.0ms preprocess, 15.9ms inference, 0.0ms loss, 1.0ms postprocess per image

Results saved to /content/runs/detect/train-3

ultralytics.utils.metrics.DetMetrics object with attributes:

ap_class_index: array([0, 1, 2])

box: ultralytics.utils.metrics.Metric object

confusion_matrix: <ultralytics.utils.metrics.ConfusionMatrix object at 0x7fd63037e580>

curves: ['Precision-Recall(B)', 'F1-Confidence(B)', 'Precision-Confidence(B)', 'Recall-Confidence(B)']

curves_results: [[array([0, 0.001001, 0.002002, 0.003003, 0.004004, 0.005005, 0.006006, 0.007007, 0.011011, 0.012012, 0.013013, 0.014014, 0.015015, 0.016016, 0.017017, 0.018018, 0.019019, 0.02002, 0.024024, 0.025025, 0.026026, 0.027027, 0.028028, 0.029029, 0.03003, 0.031031, 0.032032, 0

[7]
23
сек.

```

from ultralytics import YOLO

model_path = "runs/detect/train-3/weights/best.pt"
trained_model = YOLO(model_path)

metrics = trained_model.val()

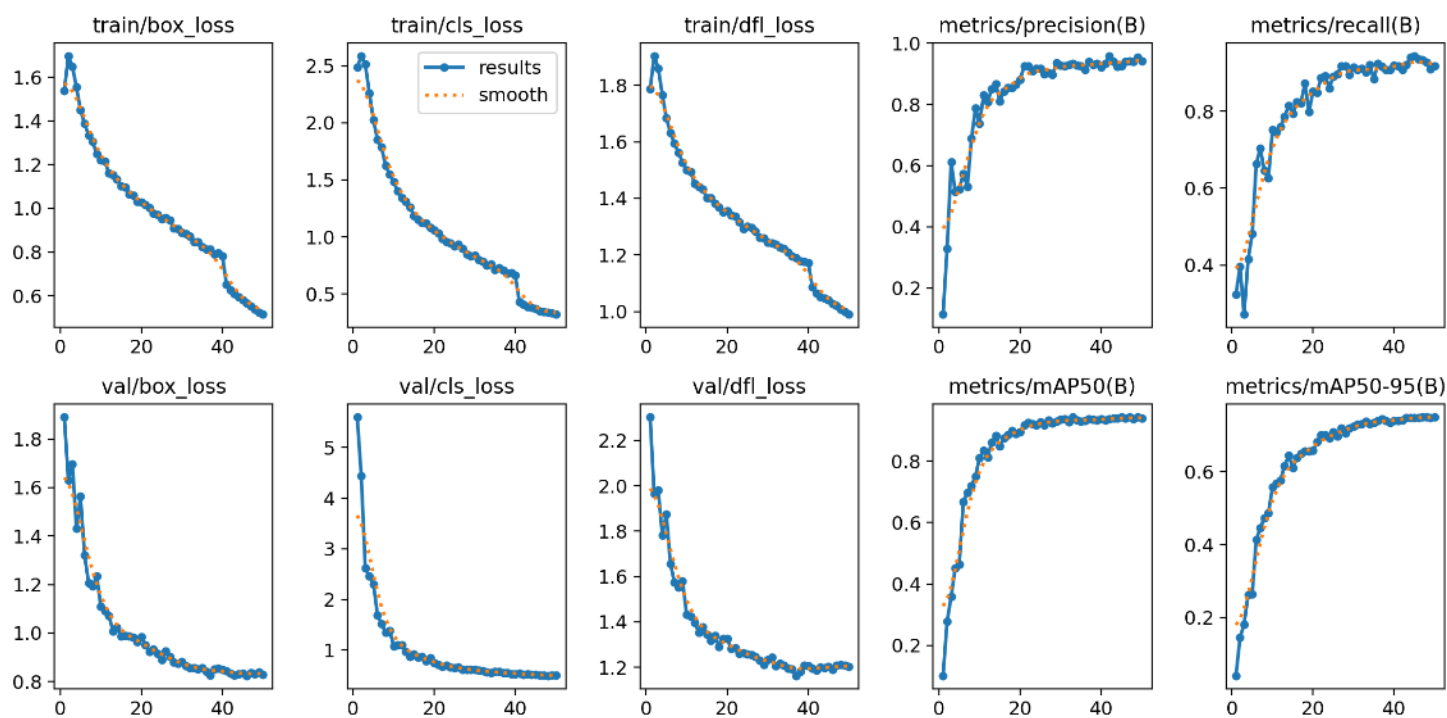
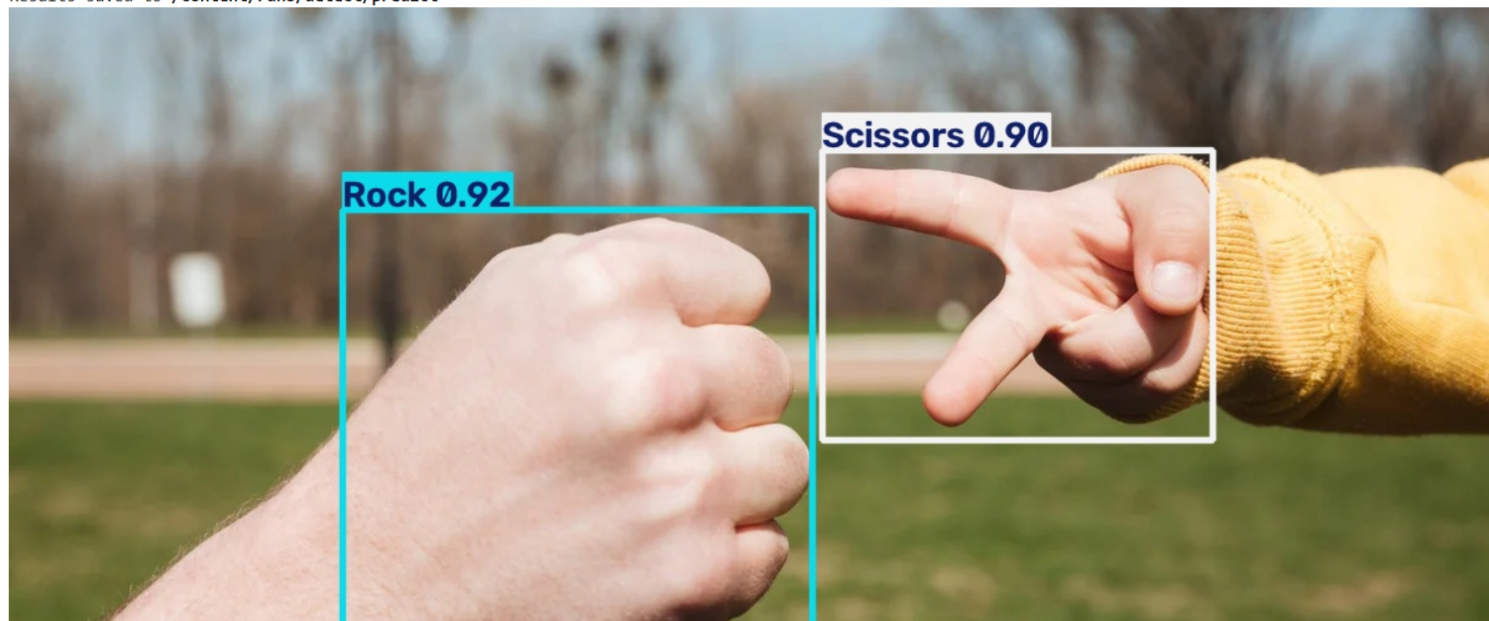
print("\nИтоговые метрики")
print("mAP50-95:", metrics.box.map)
print("mAP50:", metrics.box.map50)

... Ultralytics 8.4.159 Python-3.13.15 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)
YOLO11m summary (fused): 125 layers, 20,032,345 parameters, 0 gradients, 67.8 GFLOPs
val: Fast image access (ping: 0.0±0.0 ms, read: 764.8±318.8 MB/s, size: 29.1 KB)
val: Scanning /content/dataset/valid/labels.cache... 576 images, 238 backgrounds, 0 corrupt: 100%  576/576 105.0Mit/s 0.0s
Class Images Instances Box(P R mAP50 mAP50-95): 100%  36/36 2.0it/s 18.2s
all 576 400 0.941 0.927 0.938 0.75
Paper 132 139 0.95 0.892 0.938 0.753
Rock 121 141 0.933 0.957 0.929 0.727
Scissors 116 120 0.941 0.933 0.946 0.77
Speed: 1.9ms preprocess, 24.6ms inference, 0.0ms loss, 1.3ms postprocess per image
Results saved to /content/runs/detect/val

ИТОГОВЫЕ МЕТРИКИ
mAP50-95: 0.7499093609875088
mAP50: 0.9378247716645625

```


image 1/1 /content/test_hand.jpg: 352x640 1 Rock, 1 Scissors, 23.0ms
Speed: 2.2ms preprocess, 23.0ms inference, 1.7ms postprocess per image at shape (1, 3, 352, 640)
Results saved to /content/runs/detect/predict



Локальное тестирование:

Найдено картинок для обработки: 9

Обработка: 1.jpg...

image 1/1 D:\3\kartinki\1.jpg: 640x480 1 Rock, 241.6ms

Speed: 2.9ms preprocess, 241.6ms inference, 2.1ms postprocess per image at shape (1, 3, 640, 480)

Обработка: 2.jpg...

image 1/1 D:\3\kartinki\2.jpg: 640x480 1 Rock, 244.9ms

Speed: 2.7ms preprocess, 244.9ms inference, 1.5ms postprocess per image at shape (1, 3, 640, 480)

Обработка: 3.jpg...

image 1/1 D:\3\kartinki\3.jpg: 640x480 1 Rock, 217.5ms

Speed: 3.1ms preprocess, 217.5ms inference, 1.2ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 4.jpg...

image 1/1 D:\3\kartinki\4.jpg: 640x480 1 Paper, 258.2ms

Speed: 3.3ms preprocess, 258.2ms inference, 1.8ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 5.jpg...

image 1/1 D:\3\kartinki\5.jpg: 640x480 1 Paper, 225.0ms

Speed: 2.5ms preprocess, 225.0ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 6.jpg...

image 1/1 D:\3\kartinki\6.jpg: 640x480 1 Scissors, 209.8ms

Speed: 2.8ms preprocess, 209.8ms inference, 1.0ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 7.jpg...

image 1/1 D:\3\kartinki\7.jpg: 640x480 1 Rock, 221.1ms

Speed: 2.5ms preprocess, 221.1ms inference, 2.0ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 8.jpg...

image 1/1 D:\3\kartinki\8.jpg: 640x480 1 Rock, 223.7ms

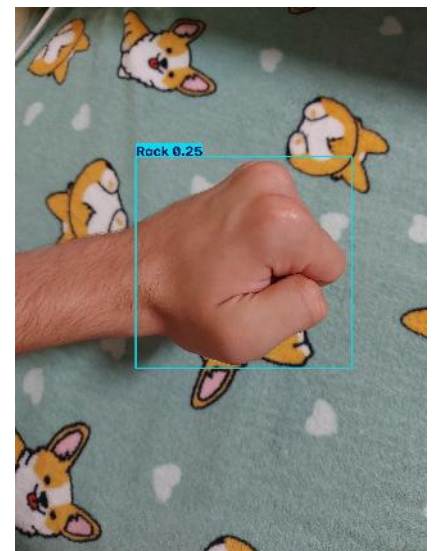
Speed: 2.7ms preprocess, 223.7ms inference, 1.4ms postprocess per image at shape (1, 3, 640, 480)
Обработка: 9.jpeg...

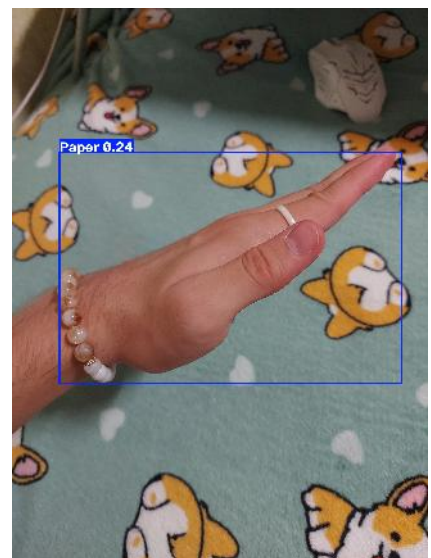
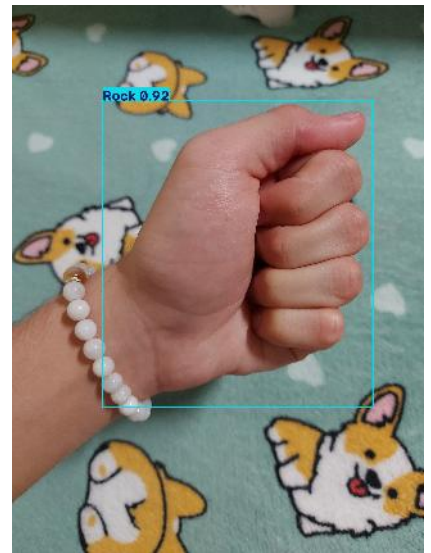
image 1/1 D:\3\kartinki\9.jpeg: 448x640 1 Rock, 1 Scissors, 320.7ms

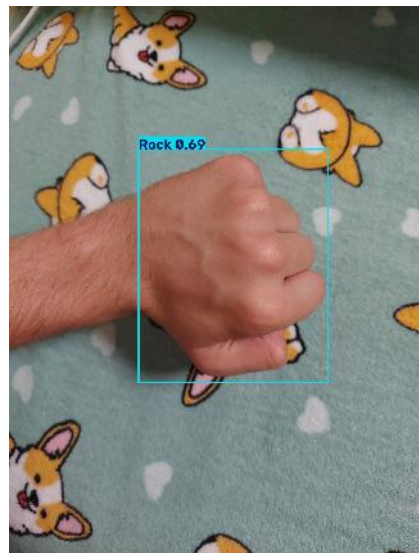
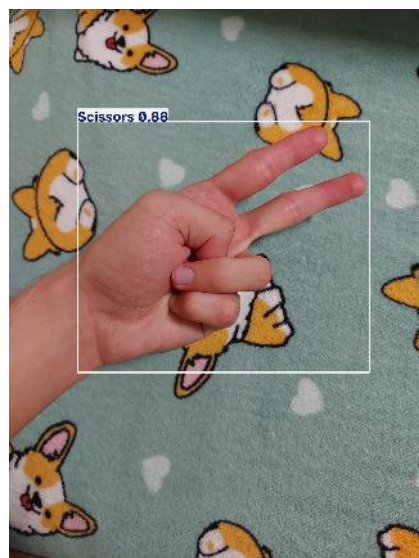
Speed: 1.8ms preprocess, 320.7ms inference, 1.6ms postprocess per image at shape (1, 3, 448, 640)

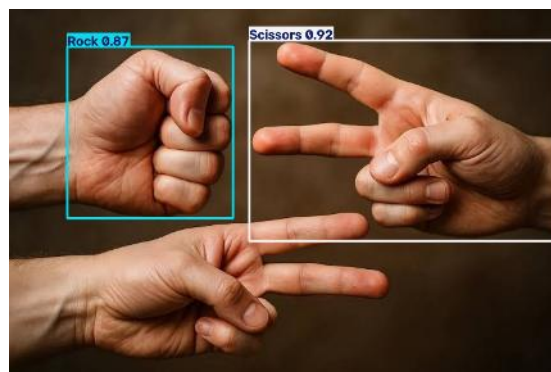
Все обработанные картинки сохранены в папку: results

Исходные картинки + результаты:









Вывод: осуществил обучение нейросетевого детектора для решения задачи обнаружения заданных объектов.